

# CATALOGGUARD LITE

## 상품 카탈로그 데이터 품질 검수 서비스

### 프로젝트 한 줄 소개

서로 다른 공급사 CSV를 JSON Profile로 표준화·검수하고 Web/CLI/S3/configured HTTP feed가 같은 ETL Core를 재사용해 PostgreSQL staging에 적재합니다. JWT viewer/operator RBAC와 Actor Audit으로 권한·실행 주체를 관리하고, Preview 승인 기반 Promotion·Audit·conflict-safe Rollback 및 Source Lineage까지 연결한 Python·FastAPI 백엔드 MVP입니다.

### 프로젝트 요약

| 항목    | 내용                                                                                                                                                                   |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 형태    | 개인 프로젝트 · 취업 포트폴리오용 MVP 고도화 (2026.08 기준)                                                                                                                             |
| 담당    | 검수 규칙, FastAPI API, JWT 인증·viewer/operator RBAC, Actor Audit, PostgreSQL, Redis·Celery, Web/CLI/S3/HTTP ETL, Promotion/Rollback/Audit, Prometheus, Streamlit, 테스트·CI |
| 핵심 기술 | Python, FastAPI, PostgreSQL, SQLAlchemy, Alembic, PyJWT, bcrypt, Redis, Celery, pandas, Streamlit, Prometheus, Airflow 3.3.0                                         |
| 운영·검증 | Docker Compose, GitHub Actions, Playwright, AWS EC2·RDS·S3 staging, Kubernetes(kind), Terraform                                                                      |

### 검증된 현재 상태

|                                         |                                      |                                                |                                       |
|-----------------------------------------|--------------------------------------|------------------------------------------------|---------------------------------------|
| 1923 passed<br>2 skipped · 5 deselected | Run 31945356223<br>5 CI jobs success | Chromium E2E 2개<br>Reject + Promotion/Rollback | Infra CI<br>K8s · Terraform · Airflow |
|-----------------------------------------|--------------------------------------|------------------------------------------------|---------------------------------------|

### 핵심 성과

상품 CSV 품질 검수와 JSON Profile 기반 표준화, normal/reject/staging 흐름을 공통 ETL Core로 구현하였습니다.

JWT viewer/operator RBAC를 서버에서 강제하고 Inspection·ETL·Promotion·Rollback의 실행 주체를 Actor Audit으로 남깁니다.

staging 데이터를 Preview·승인·hash 재검증 후 Promotion하고, 최신 상태 충돌을 보호하는 Rollback과 상품 단위 Audit을 구현하였습니다.

Web/CLI 외에도 AWS S3와 configured HTTP feed source adapter를 공통 ETL에 연결하고 최초 입력 Source Lineage를 기록합니다.

[psy0635-ctrl/catalogguard-liteCatalogGuard Lite Streamlit 앱](#) GitHub Actions에서 test, browser-e2e, kubernetes-smoke, terraform-validate, airflow-smoke 5개 job을 분리 검증합니다.

# 1. 문제 정의와 사용자 가치

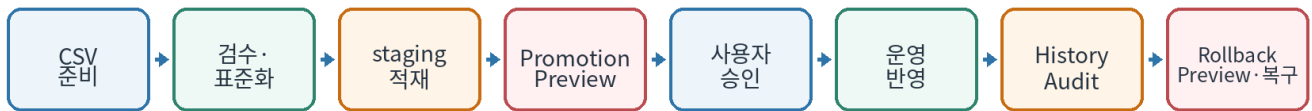
기능 수를 늘리기보다 상품 데이터가 왜 검수·표준화되어야 하고 운영 반영 전에 무엇을 확인해야 하는지부터 정의했습니다.

## 문제 정의

쇼핑몰 상품 카탈로그는 공급사와 운영 담당자마다 컬럼명·표기 방식이 달라 같은 의미가 여러 형태로 저장될 수 있습니다. 오류가 누적되면 검색·필터·재고 관리·상품 노출의 정확도가 낮아지고, 검수되지 않은 staging 데이터가 바로 운영 상품에 반영되면 변경 원인을 추적하기도 어렵습니다.

| 반복되는 문제     | 예시                    | 서비스의 대응                                     |
|-------------|-----------------------|---------------------------------------------|
| 형식 불일치      | price 에 문자, 필수값 누락    | 업로드 검증과 규칙 기반 오류 탐지                         |
| 표기 불일치      | black·블랙·BLACK        | 색상·사이즈 별칭을 표준값으로 정규화                        |
| 관계 오류       | 같은 그룹에 다른 카테고리        | product_group_id 단위로 그룹 전체 비교               |
| 공급사 구조 차이   | vendor_sku / style_id | JSON 프로필로 표준 컬럼에 매핑                         |
| 적재 추적 부족    | 표준 CSV 가 파일로만 남음      | PostgreSQL staging 배치·상품 이력 저장              |
| 운영 반영·권한 위험 | 검수 후 데이터 변경·권한 없는 실행  | RBAC + Preview Hash·승인·Audit·Rollback 으로 보호 |

## 사용자 흐름



### 사용자가 받는 결과

로그인 후 role 범위에서 검수 상태·오류 이유·수정 권장사항과 Web ETL 실행 결과, ETL 배치·상품, Promotion Preview 의 신규/수정/변경 없음 건수, 반영 후 실행 이력·상품별 Audit, Rollback Preview 의 충돌 여부와 복구 결과를 Streamlit 에서 확인할 수 있습니다.

## MVP 범위

구현 완료 — 규칙 기반 검수, 검수 이력, JWT 로그인·viewer/operator RBAC, 비동기 검수, Web/CLI 공통 Profile ETL, staging 적재·조회, Promotion Preview·승인·Audit·History, Rollback Preview·conflict 차단·transaction 복구, Streamlit UI, 브라우저 E2E.

의도적 제외 — Refresh Token·회원가입/password reset·OAuth/MFA/SSO·rate limit·세밀한 ACL/관리자 화면, streaming·증분 ETL, 대규모 동시 트래픽/부하 시험, 운영 autoscaling 은 현재 범위에 포함하지 않았습니다. Airflow 는 configured HTTP feed 의 manual trigger 만 지원하고 Terraform CI 는 실제 apply 를 수행하지 않습니다.

## 2. 시스템 구조와 책임 분리

화면·API·Authentication/RBAC·Service/Query·작업 큐·DB의 역할을 분리하고 Streamlit에서 DB를 직접 수정하지 않도록 구성했습니다.

### CatalogGuard Lite 전체 구조

로그인·JWT·검수·ETL·Promotion·Rollback을 분리하고 PostgreSQL을 공통 저장 기준으로 사용

#### 인증 + 동기·비동기 검수



#### 공급사 ETL → staging → Promotion → 필요 시 Rollback



UI는 DB에 직접 쓰지 않고 CatalogGuardApiClient → FastAPI(Auth/RBAC) → Service/Query → PostgreSQL 흐름을 사용합니다.

### 계층별 역할

| 구성                    | 주요 책임                                                              | 설계 판단                                     |
|-----------------------|--------------------------------------------------------------------|-------------------------------------------|
| Streamlit             | 검수 결과·이력, 로그인, Web ETL 실행, ETL/Promotion History·Audit·Rollback UI | DB 직접 접근 없이 CatalogGuardApiClient만 사용     |
| FastAPI               | 동기·비동기 검수, Auth, ETL 실행·조회, Promotion·Rollback API                 | 요청 검증·상태 코드·Pydantic 응답 계약                |
| Authentication / RBAC | JWT 검증·users 재조회·role/is_active 확인                                 | viewer 조회/Preview·operator 쓰기·401/403 구분  |
| Service / Query       | 검수·Promotion·Rollback 트랜잭션 / 읽기 전용 조회                              | 쓰기와 조회 책임 및 transaction boundary 분리       |
| Redis·Celery          | job 상태와 백그라운드 검수                                                   | 긴 검수를 HTTP 요청 수명과 분리                      |
| PostgreSQL            | users·검수·ETL staging·운영 상품·Promotion/rollback Run·Audit            | constraint·unique·FK·transaction으로 무결성 보장 |
| GitHub Actions        | DB/Redis/E2E/브라우저/AWS runtime 자동 검증                                | 과거 결함을 재현 가능한 회귀 검증으로 전환                  |

### 핵심 기술 판단

단일 검수 기준 — 검수 규칙은 FastAPI 서비스에서 한 번 실행하고 Streamlit은 저장된 결과를 표시하여 이중 검수를 제거하였습니다.

인증·권한 경계 — Streamlit의 버튼 비활성화는 편의 기능이고, 실제 보안은 FastAPI dependency가 JWT·현재 users 상태·role을 확인해 401/403으로 강제합니다.

읽기/쓰기 분리 — Query Service는 pagination·count 같은 읽기 전용 조회를 담당하고, 저장·Promotion·Rollback Service가 명시적 transaction boundary를 소유합니다.

공통 ETL Core — CLI와 Web API는 별도 변환 로직을 만들지 않고 같은 run\_pipeline()과 load\_standard\_csv()를 재사용해 매핑·reject·summary·SHA-256·staging 규칙을 한 곳에서 유지합니다.

### 3. 핵심 검수 기능과 결과 계약

상품 한 행의 오류뿐 아니라 패션 옵션·그룹 관계와 개인정보 의심까지 규칙 기반으로 검수하고 같은 결과 계약으로 API 와 UI 에 전달합니다.

#### 검수 범위

| 분류    | 검수 내용                                   | 대표 예시                            |
|-------|-----------------------------------------|----------------------------------|
| 입력·형식 | 필수값, category·stock·price·sale_price 형식 | price="가격문의" → 형식 오류             |
| 중복    | 상품 ID, 상품명 후보, 완전 중복                    | 동일 속성 상품 또는 ID 중복                |
| 패션 옵션 | 색상·사이즈 비표준, 옵션 조합 중복                    | 블랙 → BLACK, medium → M           |
| 그룹 관계 | 같은 product_group_id 의 카테고리 불일치          | TOP 과 BOTTOM 이 같은 그룹             |
| 가격    | 0 이하, 정상가·할인가 관계, 카테고리 이상치              | price 50,000 / sale_price 60,000 |
| 내용·보안 | 상품명-카테고리, 금지어, 개인정보 의심                  | 설명에 이메일·전화번호 포함                  |

#### 패션 데이터 정규화 예시

|                                    |                        |                         |                             |
|------------------------------------|------------------------|-------------------------|-----------------------------|
| <b>BLACK</b><br>black / 블랙 / BLACK | <b>M</b><br>medium / M | <b>XXL</b><br>2XL / XXL | <b>FREE</b><br>프리사이즈 / FREE |
|------------------------------------|------------------------|-------------------------|-----------------------------|

#### 그룹 단위 판단과 결과 계약

##### ValidationIssue

모든 규칙은 severity, product\_id, product\_group\_id, message 를 가진 공통 결과로 통일합니다. 그룹 카테고리 불일치는 다수결로 임의의 정답을 만들지 않고 불일치 사실을 참여 상품 전체에 연결하며 원본 행 순서를 유지합니다.

#### 안전한 결과 표시

내부 상세 값은 JSON 구조로 저장해 심표·작은따옴표가 포함된 값도 안전하게 전달합니다.

손상되거나 예상하지 못한 JSON 은 화면 전체를 중단시키지 않고 기본 안내 문구로 처리합니다.

다운로드 CSV 의 =, +, -, @ 시작 값은 복사본에서만 안전하게 처리해 Excel 수식 삽입 위험을 줄였습니다.

개인정보 의심 결과는 UI/ETL reject 흐름에서 마스킹하여 테스트 데이터에 실제 개인정보를 사용하지 않습니다.

## 4. 동일 CSV 재사용·검수 버전과 비동기 처리

중복 저장 방지와 불필요한 재검수 방지를 분리하고, 긴 검수는 Redis·Celery 작업으로 HTTP 요청 수명에서 분리했습니다.

### 같은 파일을 안전하게 재사용하는 방법

| 문제                | 해결 방법                            | 효과                       |
|-------------------|----------------------------------|--------------------------|
| 같은 파일 반복 업로드      | CSV bytes SHA-256                | 파일명과 무관하게 동일 내용 식별       |
| 규칙 변경 후 과거 결과 재사용 | file_sha256 + inspection_version | 버전이 바뀌면 같은 CSV 도 재검수     |
| 동시에 같은 요청         | PostgreSQL partial unique index  | 사전 조회 이후 경쟁 요청도 DB 에서 차단 |
| 같은 세션 반복 제출       | saved hash + 활성 job_id           | 불필요한 POST·GET 호출 감소      |

inspection\_version 13

검수 규칙이 변경될 때 같은 CSV 의 과거 결과가 잘못 재사용되지 않도록 버전을 올립니다. 현재 버전 13 은 기능 홍보용 숫자가 아니라 file\_sha256 과 함께 결과 재사용 정확성을 보장하는 식별자입니다.

### Redis·Celery 비동기 검수

| 항목     | 구현 내용                                   | 검증                               |
|--------|-----------------------------------------|----------------------------------|
| 작업 상태  | queued → running → succeeded / failed   | Redis DB 1 상태 조회                 |
| 작업 전달  | FastAPI → Redis broker → Celery Worker  | worker ping 및 실제 inspection task |
| 결과 저장  | 완료 후 PostgreSQL inspection_runs/results | 비동기 E2E 에서 저장 결과 확인              |
| 실패·재제출 | failed 상태 후 사용자 명시적 재제출                 | 중복 POST 방지·재시도 UI 테스트            |
| 임시 파일  | 성공·실패 후 공유 CSV 정리                       | E2E 종료 후 잔여 파일 없음                |

### 문제 해결 포인트

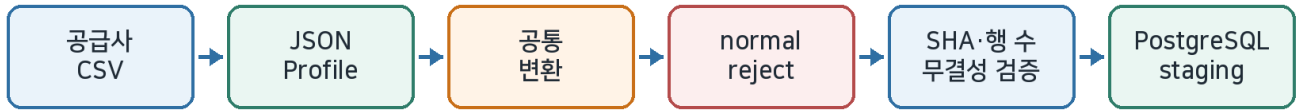
기존 방식 — 중복 여부를 저장 직전에 알았기 때문에 같은 CSV 도 전체 검수를 다시 수행했습니다.

개선 후 — 파일 검증·파싱 → hash/version 사전 조회 → 기존 결과 반환 또는 신규 검수 순서로 변경하고, 최종 중복은 DB unique constraint 로 방어했습니다.

PostgreSQL 회귀 보완 — 사전 중복 조회 SELECT 가 request Session 에 autobegin transaction 을 열어 save\_inspection\_report()의 session.begin()과 충돌하던 500 을 실제 route-level 테스트로 재현했습니다. 사전 조회를 별도 precheck\_session 으로 분리해 Service 가 유일한 transaction owner 가 되도록 유지했습니다.

## 5. Web ETL 실행과 공급사 Profile staging

operator 가 Streamlit 에서 공급사 CSV 와 허용 Profile 을 선택해 FastAPI 로 기존 ETL Core 를 실행하며, CLI 와 Web 이 같은 변환·적재 로직을 재 사용합니다.



Web ETL 실행 — operator 로그인 → CSV 업로드 → "ETL 실행 프로파일" 선택 → POST /api/v1/etl-loads → run\_web\_etl() → 기존 run\_pipeline() → load\_standard\_csv() → PostgreSQL staging → ETL History

### 합성 공급사 프로파일 비교

| 프로파일      | 원본 → 표준 컬럼                                | 설계 의미                |
|-----------|-------------------------------------------|----------------------|
| 패션 공급사 v1 | vendor_sku → product_group_id, product_id | 하나의 키를 그룹/개별 ID 에 사용 |
| 패션 공급사 v1 | discount_price → sale_price               | 선택 할인 가격을 공통 필드로 변환  |
| 마켓플레이스 v1 | style_id → product_group_id               | 상품 그룹 키를 별도로 유지      |
| 마켓플레이스 v1 | sku_code → product_id                     | 같은 그룹 아래 개별 SKU 유지   |
| 마켓플레이스 v1 | promo_price → sale_price                  | 할인 가격 관계 검수로 전달      |

### staging DB 안전장치

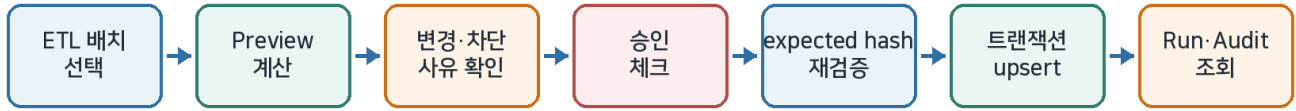
| 구성                       | 역할                            | 안전장치                                  |
|--------------------------|-------------------------------|---------------------------------------|
| etl_load_runs            | 파일·프로파일·hash·적재 행 수를 배치 단위 저장 | input hash + profile + version unique |
| catalog_products_staging | 정상 상품을 batch ID 와 연결          | stock·price·sale_price CHECK          |
| 1:N 관계                   | 배치 1 개에 상품 여러 개 연결            | Foreign Key + DB cascade              |
| 트랜잭션                     | 배치 생성·상품 저장을 하나로 처리           | 일부 실패 시 전체 rollback                   |

#### 무결성·Web 실행 안전성

표준 CSV SHA-256 과 summary JSON 의 output\_file\_sha256, 실제 행 수와 loaded\_rows 가 일치할 때만 적재합니다. Web 경로는 profile\_id 를 서버 allowlist 로 해석해 임의 filesystem path 를 차단하고, 기존 5MB 업로드 검증과 TemporaryDirectory cleanup 을 재 사용합니다. POST 실행은 operator 권한이며 동일 input hash + profile + version 은 기존 run 을 created=False 로 재사용합니다.

## 6. 안전한 Catalog Promotion 과 Audit

staging 데이터를 바로 운영 상품에 반영하지 않고 Preview·명시적 승인·Hash 재검증·트랜잭션·Audit 을 거치도록 설계했습니다.



### Promotion 안전장치

| 위험               | 구현                                      | 사용자/DB 결과                                   |
|------------------|-----------------------------------------|---------------------------------------------|
| 무엇이 바뀌는지 모름      | Preview 에서 insert/update/no-change 계산   | 상품별 변경 전·후와 반영 불가 사유 표시                     |
| Preview 후 데이터 변경 | expected_preview_hash 재계산               | 불일치 시 409 preview_stale                     |
| 실수로 즉시 반영        | operator RBAC + confirmed + 승인 checkbox | viewer 는 403 · operator 도 명시적 승인 전 UI 실행 제한 |
| 중간 실패로 일부 반영     | PostgreSQL transaction                  | 실패 시 전체 rollback                            |
| 변경 원인 추적 불가      | Promotion Run + 상품 Audit                | 이전/새 값과 관련 실행을 History 에서 조회                |

### 주요 API

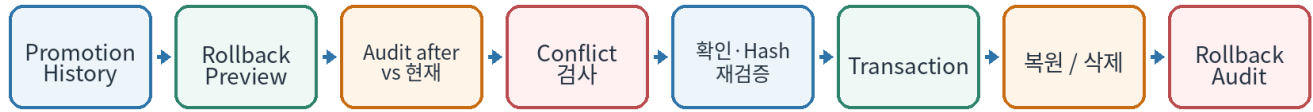
| 엔드포인트                                             | 역할                                                   |
|---------------------------------------------------|------------------------------------------------------|
| POST /api/v1/etl-loads/{run_id}/promotion-preview | viewer 이상 · 반영 예정 신규·수정·변경 없음과 Preview Hash 계산       |
| POST /api/v1/etl-loads/{run_id}/promotions        | operator · confirmed + expected_preview_hash 로 실제 반영 |
| GET /api/v1/catalog-promotions                    | viewer 이상 · Promotion 실행 이력 목록·상태 필터·페이지네이션          |
| GET /api/v1/catalog-promotions/{id}               | viewer 이상 · 실행 상세                                    |
| GET /api/v1/catalog-promotions/{id}/audits        | viewer 이상 · 상품별 변경 Audit 조회                          |

#### 용어 주의

이 프로젝트에서 Promotion 은 할인 행사가 아니라 ETL staging 의 검수된 상품을 운영 카탈로그에 반영하는 과정입니다.

## 7. 안전한 Catalog Promotion Rollback

잘못 반영된 Promotion 을 과거 값으로 무조건 덮어쓰지 않고, 현재 상태와 Audit 을 비교해 안전할 때만 하나의 transaction 으로 복구하도록 설계했습니다.



### Rollback 안전장치

| 위험                        | 구현                                               | 결과                                      |
|---------------------------|--------------------------------------------------|-----------------------------------------|
| 최신 변경을 덮어쓸 위험             | Audit after 와 현재 business field 비교               | 다르면 rollback_conflict 로 실행 차단           |
| Rollback Preview 후 데이터 변경 | rollback preview hash 실행 직전 재계산                  | 불일치 시 409 preview_stale                 |
| 실수로 즉시 복구                 | operator RBAC + confirmation 서버 검증 + UI checkbox | viewer 는 403 · 승인 없이는 DB 변경 전 400 으로 차단 |
| 중간 실패로 일부만 복구             | 단일 PostgreSQL transaction                        | 실패 시 전체 원복(All-or-Nothing)              |
| 같은 Promotion 중복 Rollback  | FOR UPDATE + partial unique index                | 성공 Rollback 은 1 회만 허용                   |

### Rollback API 와 복원 규칙

| 엔드포인트                                                 | 역할                                                           |
|-------------------------------------------------------|--------------------------------------------------------------|
| POST /api/v1/catalog-promotions/{id}/rollback-preview | viewer 이상 · 복원/삭제 대상, conflict 여부와 Rollback Preview Hash 계산  |
| POST /api/v1/catalog-promotions/{id}/rollback         | operator · confirmation + expected_preview_hash 재검증 후 원자적 복구 |
| INSERT 변경 복구                                          | 현재 상태가 원본 Audit after 와 같을 때 운영 상품 삭제                        |
| UPDATE 변경 복구                                          | 원본 Audit before_data 로 비교 대상 비즈니스 필드 복원                      |
| Rollback Run / Change                                 | 실행 상태와 상품별 before/after/action 기록, 원본 Promotion Audit 보존     |

#### 검증된 결과

실제 PostgreSQL 에서 INSERT 삭제, UPDATE 복원, conflict·중복 차단, 강제 실패 원자성, 원본 Audit 보존을 검증했습니다. 최신 GitHub Actions Run 31945356223 에서 1923 passed · 2 skipped · 5 deselected · 0 failed 로 전체 회귀를 재검증했습니다.



## 8. 로그인·Web ETL·Promotion 이력 UI 와 Rollback UI

Streamlit 사이드바 로그인·로그아웃과 role 기반 UI 를 추가하고 Web ETL 실행, 읽기 전용 History/Audit, Rollback UI 를 연결했습니다. UI 제한과 별개로 FastAPI 가 JWT 와 RBAC 를 최종 검증합니다.

### 조회·상태 관리

| 기능                | 구현 판단                                                           | 검증                                                             |
|-------------------|-----------------------------------------------------------------|----------------------------------------------------------------|
| 로그인·권한 UI         | username/password → JWT 를 session_state 에 저장 · logout · role 표시 | login/me·401/403·viewer/operator 테스트 · 서버가 쓰기 권한 최종 차단         |
| Web ETL 실행        | CSV 업로드·"ETL 실행 프로필" 선택·명시적 버튼 POST                             | operator 권한 · API/Client/PostgreSQL/AppTest · History cache 갱신 |
| ETL 목록·검색         | 파일명·프로필 AND 검색, SQL LIMIT/OFFSET                                | 특수문자 escape·정렬·count 일치                                        |
| ETL 상세·상품         | SHA-256·nullable 처리, 상품 페이지                                     | 404·request ID·호출 횟수 AppTest                                   |
| Promotion History | 상태 필터·목록/상세·Audit 분리 조회                                         | 목록에서 Audit N+1 회피                                              |
| Rollback UI       | Preview·conflict·confirmation·실행 상태 관리                          | 서버 hash 재검증·PostgreSQL 통합 검증                                   |
| Cache 갱신          | Promotion 성공 시 history cache 만 무효화                              | 성공 직후 최신 run 이 즉시 표시                                           |
| 반응형 UI            | 핵심 버튼·상태 계층·간격 정리                                               | Playwright 1440px / 390px 실제 브라우저 확인                           |

### Chromium Browser E2E

|                                               |                                          |                                       |                                         |
|-----------------------------------------------|------------------------------------------|---------------------------------------|-----------------------------------------|
| <b>2 개 시나리오</b><br>GitHub Actions browser-e2e | <b>실제 Chromium</b><br>버튼·checkbox·페이지 이동 | <b>DB 최종 검증</b><br>운영 상품·Run·Audit 확인 | <b>조회 안전성</b><br>History/Audit 전후 DB 불변 |
|-----------------------------------------------|------------------------------------------|---------------------------------------|-----------------------------------------|

Promotion E2E — synthetic operator 로 Streamlit 로그인 후 reject/promotion batch fixture 를 준비하고 Preview → 승인 → 실행 → History/Audit 까지 조작한 뒤 PostgreSQL 최종 상태를 확인합니다.

실제 결함 수정 — Web ETL selectbox 가 기존 검색 필드와 같은 accessible label 을 사용해 Playwright strict-mode violation 이 발생했습니다. selector 를 .first()로 우회하지 않고 신규 label 을 "ETL 실행 프로필"로 분리해 기존 E2E 를 회복했습니다.

읽기 전용 보장 — History 와 Audit 조회 전후 DB 상태가 동일함을 확인하여 조회 기능이 운영 데이터를 변경하지 않도록 검증했습니다.

범위 구분 — Web ETL 핵심 실행은 API·Client·PostgreSQL 통합 테스트와 Streamlit AppTest 로 검증했으며, CSV file-upload 자체를 처음부터 끝까지 조작하는 전용 Browser E2E 는 아직 없습니다.

### API Client 방어

CatalogGuardApiClient

login()/get\_current\_user()/set\_access\_token()으로 Bearer JWT 를 세션 헤더에 적용하고 401·403 을 도메인 예외로 구분합니다. 기존 응답 schema 와 lowercase SHA-256 Preview Hash 검증도 유지하며, Streamlit 은 이 Client 만 사용해 SQL 이나 운영 상품 update/delete 를 직접 실행하지 않습니다.

## 9. CI·Docker Runtime 검증과 성능 판단

테스트 개수보다 실제 서비스를 연결한 회귀 검증과 과거 장애를 CI에서 다시 잡을 수 있는지를 중요하게 보았습니다.

### GitHub Actions 자동 검증

| 단계                | 검증 범위                                                                                                                               |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| DB·Queue          | PostgreSQL 18·Redis 7.4 서비스 시작, Alembic upgrade head(users 포함)                                                                      |
| 회귀 테스트            | E2E 제외 pytest 1923 passed · Auth/RBAC · Web ETL · Sync Inspection 실제 PostgreSQL regression                                          |
| Rollback DB       | PostgreSQL 에서 INSERT 삭제·UPDATE 복원·conflict·중복·원자성·Audit 보존 검증                                                                       |
| 비동기 E2E           | operator login/JWT → job 제출·polling·결과 저장·동일 CSV 재사용·임시 파일 정리                                                                       |
| Streamlit         | startup smoke /_stcore/health HTTP 200 · 로그인 게이트 회귀 포함                                                                              |
| AWS Runtime smoke | Dockerfile.aws build → 핵심 imports → UID 10001 → migration → 기본 CMD Uvicorn → container /health; 별도 FastAPI readiness /health·/ready |
| Browser E2E       | operator 로그인 후 Playwright Chromium 기준 2 개 시나리오                                                                                      |

### 실제 장애 → 재발 방지

실제 장애 → 재발 방지: 테스트 수보다 실제 실행 경로를 확인했습니다

- 1) Web ETL — image build 는 성공했지만 runtime import 에서 etl package 누락이 드러나 Dockerfile.aws COPY 를 보완했습니다.
- 2) Sync Inspection — 1262 tests 가 통과하던 상태에서도 실제 PostgreSQL route 에서 사전 SELECT autobegin 과 save\_inspection\_report()의 begin 이 충돌해 500 이 발생했습니다. 별도 precheck Session 과 route-level PostgreSQL regression 5 개로 테스트 공백을 보완했습니다.

### PostgreSQL 성능 판단

| 기존 PostgreSQL 18 benchmark | 결과                                                                         |
|----------------------------|----------------------------------------------------------------------------|
| 데이터 규모                     | inspection_runs 10,000 · inspection_results 100,000                        |
| 측정 방식                      | EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON), warmup 2 회 + 측정 7 회               |
| 주요 조회 중앙값                  | 동일 CSV 0.018 ms · 목록 0.044 ms · count 0.872 ms · 상세 0.016 ms · 결과 0.029 ms |
| 판단                         | 현재 측정 범위에서 병목 근거 없음 → 불필요한 index/migration 추가하지 않음                         |

※ 성능 수치는 기존 저장 benchmark 를 재검토한 결과이며 운영 트래픽 성능을 보증하지 않습니다. ETL·Promotion history 는 향후 전용 PostgreSQL 데이터가 충분히 쌓인 뒤 별도 측정 후보입니다.

## 10. 검증 결과, 현재 한계와 30 초 소개

현재 구현된 범위와 아직 검증하지 않은 부분을 분리하여 과장 없이 제출할 수 있도록 정리했습니다.

### 검증 결과

| 검증 항목              | 현재 기준                                                                             | 해석                                                                   |
|--------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------|
| 최근 전체 pytest       | 1923 passed · 2 skipped · 5 deselected                                            | Auth/RBAC·Sync Inspection·Rollback·Web ETL PostgreSQL 통합 포함 전체 회귀 기준 |
| GitHub Actions     | Run 31945356223 · test / browser-e2e success                                      | DB·Redis·Auth/RBAC·Async E2E·Streamlit·AWS runtime·Chromium 검증       |
| AWS Docker Runtime | image build/import/UID 10001/migration/Uvicorn /health · readiness /health /ready | 실제 AWS 재배포가 아닌 Ubuntu runner runtime/readiness smoke                 |
| 실제 AWS staging     | 2026-07 EC2·RDS·TLS·Docker·FastAPI·Streamlit 검증 경험                                | 비용 절감으로 리소스 중지, 최신 변경 재배포는 미 실행                                      |

### 현재 한계

| 구분         | 현재 상태                                                                                                                           |
|------------|---------------------------------------------------------------------------------------------------------------------------------|
| 인증 확장      | JWT Access Token·viewer/operator RBAC 및 Actor Audit 구현 · Refresh Token·가입/password reset·OAuth/MFA/SSO·rate limit·세밀한 ACL 은 미구현 |
| 운영 비동기     | Redis·Celery 는 로컬·CI E2E 검증, 운영 배포는 미검증                                                                                         |
| 데이터 유입     | 합성 공급사 프로필 2 종·수동 선택·CSV 중심                                                                                                     |
| ETL 실행     | Streamlit Web ETL + CLI + S3 + configured HTTP feed 지원 · Web 업로드는 동기·단일 CSV · Airflow HTTP feed 는 manual trigger                |
| 대용량        | streaming·중분 ETL·동시 트래픽 부하 측정 미지원                                                                                               |
| AWS 최신 재검증 | AWS image 는 CI runtime 검증, 최근 코드 기준 EC2·RDS 재배포는 미실행                                                                            |

### 30 초 소개

#### 면접 답변

CatalogGuard Lite 는 공급사 상품 CSV 를 JSON Profile 로 표준화하고 품질 오류를 검수하는 Python·FastAPI 데이터 품질 백엔드 프로젝트입니다. JWT viewer/operator RBAC 와 Actor Audit 으로 권한과 실행 주체를 서버에서 관리하고, Web/CLI/S3/HTTP 입력이 공통 ETL Core 를 재사용해 PostgreSQL staging 에 적재됩니다. 이후 Preview Hash 기반 Promotion·Audit·History 와 conflict-safe Rollback 으로 운영 변경을 보호하며, GitHub Actions 5 개 job 에서 일반 테스트·Chromium E2E·Kubernetes·Terraform·Airflow 를 분리 검증합니다.

### 제출 기준

기준일 2026-08-17 · HEAD ffa20af · GitHub Actions 31945356223 5/5 success · 1923 passed · 2 skipped · 5 deselected · 0 failed · INSPECTION\_VERSION 13 · profile\_version 2

## 11. 최신 운영·인프라 확장과 Source Lineage

기존 FastAPI·PostgreSQL ETL Core 를 바꾸지 않고 입력 경로와 실행 환경을 확장했습니다. 기능을 새로 복제하기보다 source adapter·orchestration·deployment layer 를 분리해 같은 품질 규칙과 staging 정책을 재사용하는 것을 우선했습니다.

| 영역               | 현재 구현                                                                                   | 검증 근거                                                                       |
|------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Actor Audit      | Inspection·ETL·Promotion·Rollback Run 에 actor_user_id + actor_username snapshot 저장      | JWT current_user 에서만 기록, 사용자 삭제 시 FK NULL + username snapshot 보존            |
| Source Lineage   | initial_source_type: upload / s3 / http_feed / cli / unknown, 안전한 initial_source_ref 저장 | Alembic head 20260813_0013, URL/token·bucket 민감정보 원문 미저장                    |
| AWS S3 ingestion | EC2 Instance Role 의 s3:GetObject 최소 권한으로 private S3 합성 CSV 를 기존 ETL→RDS staging 에 적재    | 정확한 prefix 제한, 정적 access key 미주입, idempotency/Actor Audit 검증                |
| Prometheus       | /metrics 에 HTTP count·latency·status 및 Web ETL 처리 metric 노출                             | route template label 로 cardinality 제어, feature flag off 시 no-op             |
| Kubernetes       | 기존 Dockerfile.aws 재사용, Migration Job 과 API Deployment 책임 분리                             | kind 에서 PostgreSQL→Migration Job→API rollout, /health·/ready HTTP 200       |
| Terraform        | 검증한 AWS staging 구조를 코드화: EC2/RDS/SG/SSM                                                 | EC2 inbound 0, RDS SG 참조 5432, non-public/encrypted, mock test; CI apply 없음 |
| Airflow 3.3.0    | configured HTTP feed → 기존 ETL/staging 을 manual DAG 로 orchestration                      | metadata DB 분리, transient retry, 동일 bytes idempotency·lineage 검증            |

**핵심 판단:** AWS·Kubernetes·Terraform·Airflow 를 프로젝트의 목적처럼 앞세우지 않고, 상품 데이터 품질과 안전한 반영 흐름을 운영 환경에서도 재현·검증하기 위한 보조 계층으로 사용했습니다.

## 12. 최신 정책·CI·데모 제출 기준

| 항목                  | 최신 기준                                                                                                         |
|---------------------|---------------------------------------------------------------------------------------------------------------|
| main HEAD           | ffa20af05377a15bb450395be19a39c0a9e71342                                                                      |
| GitHub Actions      | Run 31945356223 · test / browser-e2e / kubernetes-smoke / terraform-validate / airflow-smoke 5/5 success      |
| 일반 pytest           | 1923 passed · 2 skipped · 5 deselected · 0 failed                                                             |
| Chromium E2E        | 2 개 시나리오: ETL reject·민감값 masking + Promotion→Audit→Rollback→Rollback Change Audit; PostgreSQL 최종 상태 확인        |
| 검수 버전               | INSPECTION_VERSION = 13                                                                                       |
| ETL Profile         | sample profile_version = 2                                                                                    |
| 카테고리/size           | TOP/BOTTOM/OUTER/SHOES 는 size 필수, BAG 은 선택. BAG 의 빈 값과 FREE 는 모두 허용하되 서로 다른 값으로 유지                            |
| Unknown Size Policy | 사전에 없는 size 는 그 값만으로 오류/주의를 만들지 않고 미판정; fuzzy matching 미사용                                                    |
| 로컬 Demo             | PostgreSQL·Redis·FastAPI /health·/ready(database ok)·Streamlit·Chromium 2 개 시나리오 확인, 합성 fixture 기반 Demo Ready |

### 현재 한계

Refresh Token·회원가입/reset·OAuth/MFA/SSO·rate limit·세밀한 ACL/관리자 기능은 구현하지 않았습니다.  
 Airflow 는 configured HTTP feed 의 manual trigger 범위이며 스케줄 운영·다중 DAG 운영 자동화는 현재 범위가 아닙니다.  
 Terraform 은 validate 와 mock provider test 까지 자동 검증하며 CI 에서 실제 AWS apply 를 수행하지 않습니다.  
 Kubernetes 는 kind smoke 기준이며 production cluster·autoscaling·장애 복구 운영까지 검증한 것은 아닙니다.  
 현재 성능 자료는 제한된 합성 benchmark 이며 대규모 동시 트래픽/SLA 를 보증하지 않습니다.